Lab 3: Sequential Circuit Design

Moore and Mealy Models for Sequence Detection

Alexandria University
Faculty of Engineering
Computer and Systems Engineering Department

Course: Digital Systems Design (CSE 232)

Submitted by:

Youssef Diaa Abdelmohsen Mahmoud Elghanam	24010851
Mohamed Hamed Mohamed	24010593
Mohamed Sabr Abbas Elsaid	24010617
Abdelrahman Mohamed Mohamed Seif Elnasr Mohamed	24010397
Omar Mohamed Abdelmohsen Mostafa Elsaid	24010467

May 16, 2026

Contents

1 Problem Statement

The goal of this lab is to design a sequential circuit with one input X and one output Z. The circuit should be implemented using both Moore and Mealy models.

Output Condition: $Z = 1$ if and only if: 1. The total number of 1's received is divisible by 4. 2. The total number of 0's received is an odd number.

Events should take place at each rising edge of the clock (Clk).

1.1 Interface

Signal	Type	Description
X	Input	Input of the state machine
Clk	Input	Clock to synchronize events
Z	Output	Output of the state machine

2 State Machine Design

To track the conditions, we need to maintain: - The count of 1's modulo 4 (0, 1, 2, 3). - The parity of the count of 0's (Even, Odd).

This results in $4 \times 2 = 8$ states.

2.1 State Definitions

State Name	Ones mod 4	Zeros parity	Z (Moore)
S_0_EVEN	0	even	0
S_0_ODD	0	odd	1
S_1_EVEN	1	even	0
S_1_ODD	1	odd	0
S_2_EVEN	2	even	0
S_2_ODD	2	odd	0
S_3_EVEN	3	even	0
S_3_ODD	3	odd	0

2.2 State Diagrams

The following figures show the state transitions for both models.

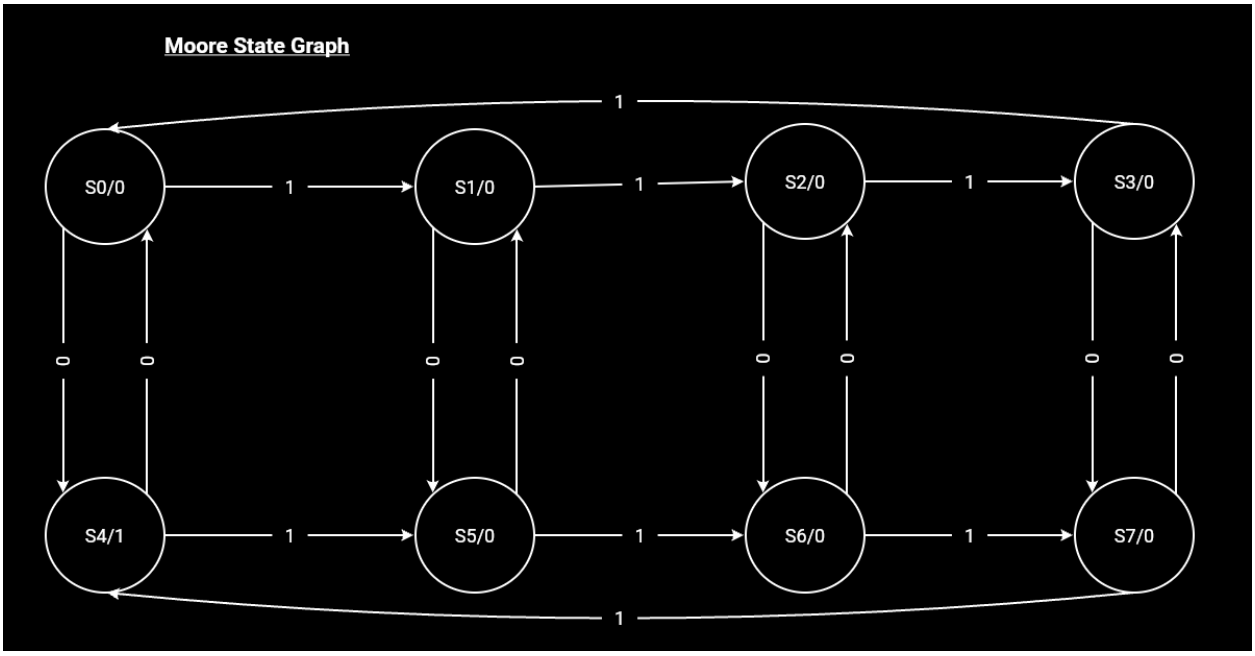

Figure 1: Moore State Graph

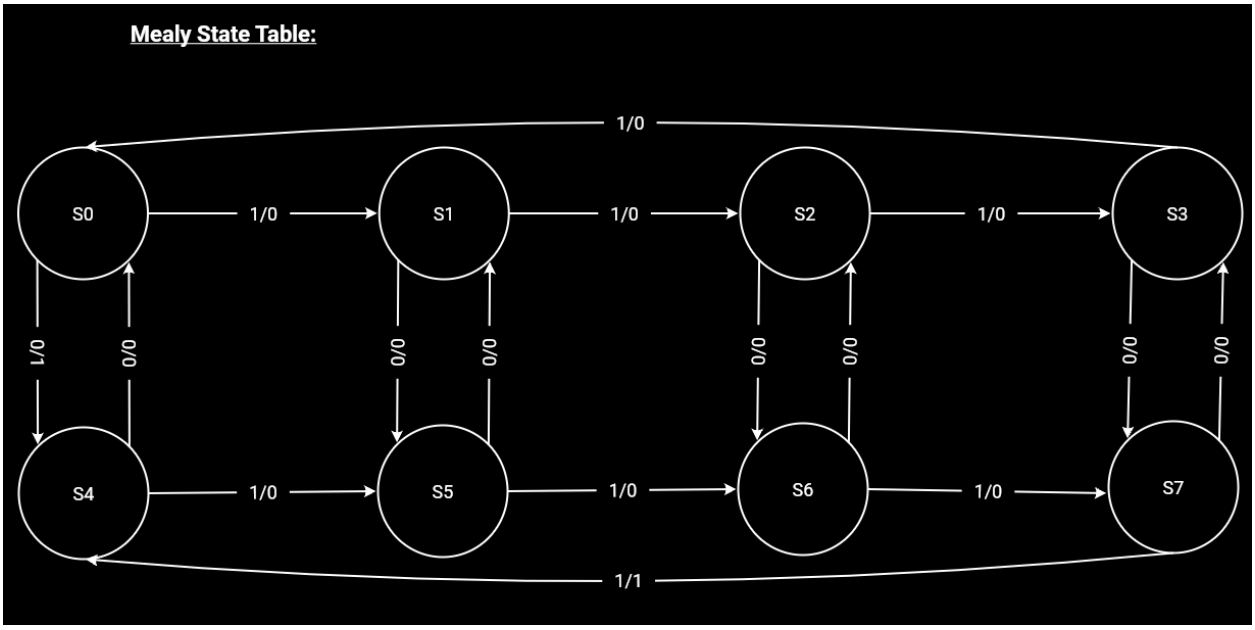

Figure 2: Mealy State Graph

2.3 Transition and State Tables

The following tables and Karnaugh maps provide the detailed logical design for both the Moore and Mealy implementations.

2.3.1 Moore Model Design

Moore State Table:

Present State	Next State		Output(Z):
	X = 0	X = 1	
S0	S4	S1	0
S1	S5	S2	0
S2	S6	S3	0
S3	S7	S0	0
S4	S0	S5	1
S5	S1	S6	0
S6	S2	S7	0
S7	S3	S4	0

Figure 3: Moore State Table

Moore Transition Table:

Q2	Q1	Q0	X	Q2+	Q1+	Q0+	J2	K2	J1	K1	J0	K0	Z
0	0	0	0	1	0	0	1	X	0	X	0	X	0
0	0	0	1	0	0	1	0	X	0	X	1	X	0
0	0	1	0	1	0	1	1	X	0	X	X	0	0
0	0	1	1	0	1	0	0	X	1	X	X	1	0
0	1	0	0	1	1	0	1	X	X	0	0	X	0
0	1	0	1	0	1	1	0	X	X	0	1	X	0
0	1	1	0	1	1	1	1	X	X	0	X	0	0
0	1	1	1	0	0	0	0	X	X	1	X	1	0
1	0	0	0	0	0	0	X	1	0	X	0	X	1
1	0	0	1	1	0	1	X	0	0	X	1	X	1
1	0	1	0	0	0	1	X	1	0	X	X	0	0
1	0	1	1	1	1	0	X	0	1	X	X	1	0
1	1	0	0	0	1	0	X	1	X	0	0	X	0
1	1	0	1	1	1	1	X	0	X	0	1	X	0
1	1	1	0	0	1	1	X	1	X	0	X	0	0
1	1	1	1	1	0	0	X	0	X	1	X	1	0

Figure 4: Moore Transition Table

2.3.2 Mealy Model Design (Bonus)

Mealy State Table:

Present State	Next State		Output(Z):	
	X = 0	X = 1	X = 0	X = 1
S0	S4	S1	1	0
S1	S5	S2	0	0
S2	S6	S3	0	0
S3	S7	S0	0	0
S4	S0	S5	0	0
S5	S1	S6	0	0
S6	S2	S7	0	0
S7	S3	S4	0	1

Figure 5: Mealy State Table

Mealy Transition Table:

Q2	Q1	Q0	X	Q2+	Q1+	Q0+	J2	K2	J1	K1	J0	K0	Z
0	0	0	0	1	0	0	1	X	0	X	0	X	1
0	0	0	1	0	0	1	0	X	0	X	1	X	0
0	0	1	0	1	0	1	1	X	0	X	X	0	0
0	0	1	1	0	1	0	0	X	1	X	X	1	0
0	1	0	0	1	1	0	1	X	X	0	0	X	0
0	1	0	1	0	1	1	0	X	X	0	1	X	0
0	1	1	0	1	1	1	1	X	X	0	X	0	0
0	1	1	1	0	0	0	0	X	X	1	X	1	0
1	0	0	0	0	0	0	X	1	0	X	0	X	0
1	0	0	1	1	0	1	X	0	0	X	1	X	0
1	0	1	0	0	0	1	X	1	0	X	X	0	0
1	0	1	1	1	1	0	X	0	1	X	X	1	0
1	1	0	0	0	1	0	X	1	X	0	0	X	0
1	1	0	1	1	1	1	X	0	X	0	1	X	0
1	1	1	0	0	1	1	X	1	X	0	X	0	0
1	1	1	1	1	0	0	X	0	X	1	X	1	1

Figure 6: Mealy Transition Table

2.3.3 Karnaugh Maps and Logic Equations

The logic equations were derived using the following Karnaugh maps:

J2:

Q2Q1/Q0X	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	X	X	X	X

J2 = (NOT X);

K2:

Q2Q1/Q0X	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	1	0	0	1
10	1	0	0	1

K2 = (NOT X);

J1:

Q2Q1/Q0X	00	01	11	10
00	0	0	1	0
01	X	X	X	X
11	X	X	X	X
10	0	0	1	0

J1 = Q0 AND X

Figure 7: K-maps Page 1

K1:

Q2Q1/Q0X	00	01	11	10
00	X	X	X	X
01	0	0	1	0
11	0	0	1	0
10	X	X	X	X

K1 = Q0 AND X

J0:

Q2Q1/Q0X	00	01	11	10
00	0	1	X	X
01	0	1	X	X
11	0	1	X	X
10	0	1	X	X

J0 = X

K0:

Q2Q1/Q0X	00	01	11	10
00	X	X	1	0
01	X	X	1	0
11	X	X	1	0
10	X	X	1	0

K0 = X

Figure 8: K-maps Page 2

Z(MOORE):

Q2Q1/Q0X	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	1	0	0

Z = Q2 AND (NOT Q1) AND (NOT Q0)

Z(MEALY):

Q2Q1/Q0X	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	1	0
10	0	0	0	0

Z = ((NOT Q2) AND (NOT Q1) AND (NOT Q0) AND (NOT X)) or (Q2
AND Q1 AND Q0 AND X)

Figure 9: K-maps Page 3

3 Moore Model Implementation

The Moore model determines the output Z based only on the current state.

3.1 VHDL Implementation

The implementation uses a three-process approach: 1. **State Register:** Synchronous update of the state. 2. **Next State Logic:** Combinational logic to determine the next state. 3. **Output Logic:** Combinational logic to determine the output.

```
-- State Register
state_register: process(Clk)
begin
    if rising_edge(Clk) then
        current_state <= next_state;
    end if;
end process;

-- Output Logic (Moore)
output_logic: process(current_state)
begin
    case current_state is
        when S_0_ODD =>
            Z <= '1';
        when others =>
            Z <= '0';
    end case;
end process;
```

4 Mealy Model Implementation (Bonus)

In the Mealy model, the output Z depends on both the current state and the current input X. This allows the output to react immediately to changes in X before the next clock edge.

4.1 VHDL Implementation

The Mealy model often combines next state and output logic into a single combinational process.

```
mealy_logic: process(current_state, X)
begin
    Z <= '0'; -- Default
    case current_state is
        when S_0_EVEN =>
            if X = '0' then
```

```
                next_state <= S_0_ODD;
                Z <= '1'; -- React immediately
            else
                next_state <= S_1_EVEN;
            end if;
        -- ... other states ...
        when S_3_ODD =>
            if X = '0' then
                next_state <= S_3_EVEN;
            else
                next_state <= S_0_ODD;
                Z <= '1'; -- React immediately
            end if;
    end case;
end process;
```

5 Moore vs Mealy Comparison

5.1 Timing Differences

The key difference observed during simulation is the response time: - **Moore:** The output Z changes only on the rising edge of the clock after the input X has changed. - **Mealy:** The output Z changes immediately when the input X changes (combinational path).

Feature	Moore Model	Mealy Model
Output depends on	Current state only	State and Input
Hardware response	One clock cycle delay	Immediate (asynchronous)
Stability	Stable, synchronous	Potential for glitches

5.2 Simulation Results

A comparison testbench was used to verify the functional equivalence and timing differences.

```
-- From tb_moore_mealy_comparison.vhd
report ">>> Changing X to 0 - CRITICAL MOMENT <<<";
X_tb <= '0';
wait for 1 ns;
report "Moore Z = " & std_logic'image(Z_moore) & " (Waiting for clock)";
report "Mealy Z = " & std_logic'image(Z_mealy) & " (Already 1!)";
```

6 Simulation Results

The functionality of both the Moore and Mealy models was verified through timing simulations. The following waveforms demonstrate the circuit's response to various input sequences.

6.1 Moore Model Simulation

The Moore model simulation confirms that the output Z is synchronized with the rising edge of the clock. Z only becomes high when the current state satisfies the condition (0 ones mod 4, odd zeros).

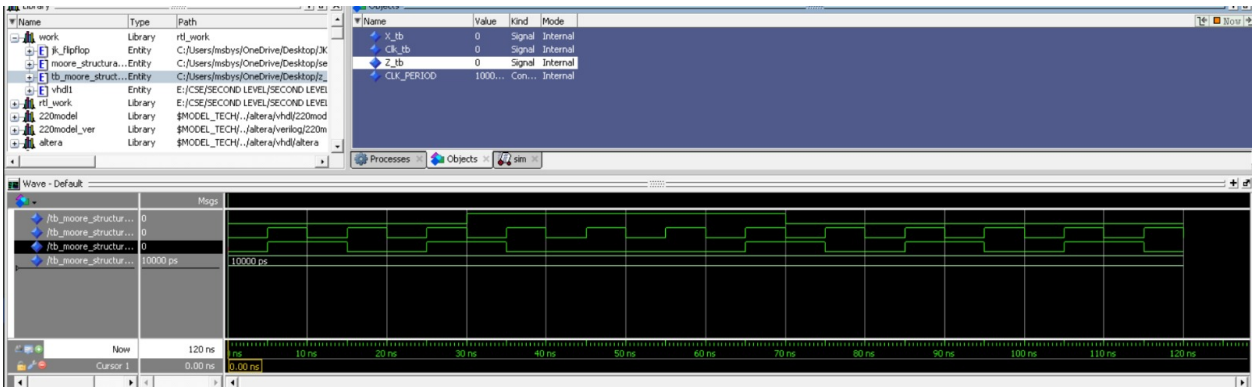

Figure 10: Moore Model Waveform

6.2 Mealy Model Simulation

The Mealy model simulation highlights the asynchronous nature of the output. As seen in the waveform, Z can react to changes in the input X within the same clock cycle, providing a faster detection of the target sequence.

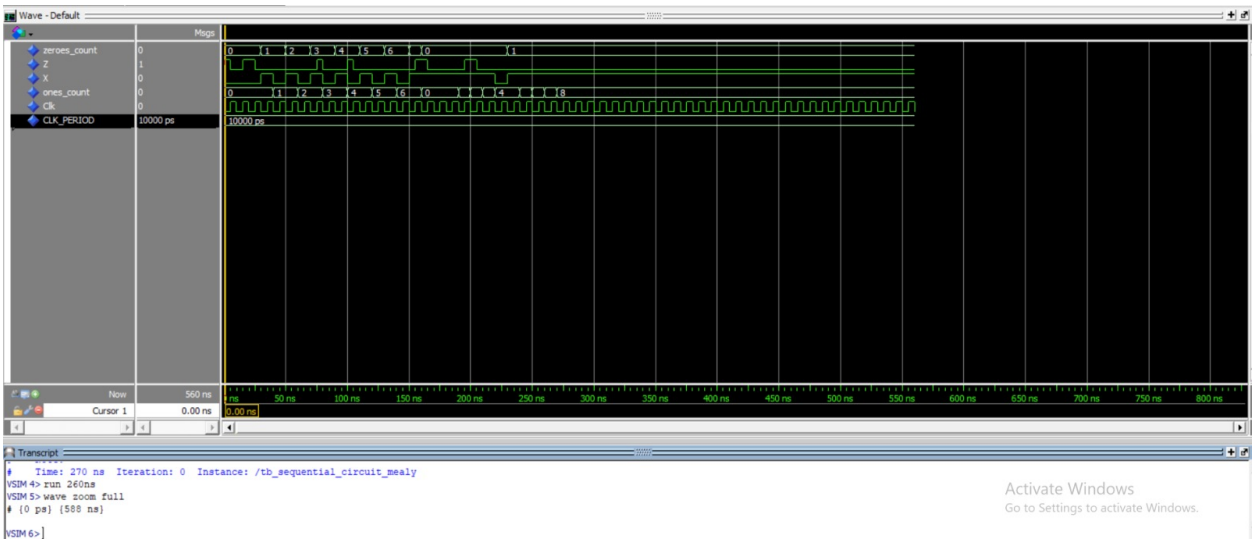

Figure 11: Mealy Model Waveform

7 Design Decisions and Assumptions

1. **State Encoding:** Enumerated types were used in VHDL for better readability and to let the synthesizer optimize the encoding.

2. **Reset:** A power-on initialization was assumed (`signal ... := S_0_EVEN`), as no explicit reset signal was required in the interface.
3. **Clock:** All transitions occur on the rising edge of the clock.

8 Conclusion

The lab successfully demonstrated the design of sequential circuits using both Moore and Mealy architectures. The requirement of tracking ones (mod 4) and zeros (parity) was met using an 8-state FSM. The Mealy model provided a faster response time, while the Moore model offered more stable, clock-synchronized outputs.

9 Appendix: Source Code

9.1 Moore Architecture

```vhdl
-- sequential_circuit_moore.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sequential_circuit_moore is
    Port (
        X   : in  STD_LOGIC;
        Clk : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end sequential_circuit_moore;

architecture Behavioral of sequential_circuit_moore is
    type state_type is (
        S_0_EVEN, S_0_ODD, S_1_EVEN, S_1_ODD,
        S_2_EVEN, S_2_ODD, S_3_EVEN, S_3_ODD
    );
    signal current_state, next_state : state_type := S_0_EVEN;
begin
    state_register: process(Clk)
    begin
        if rising_edge(Clk) then
            current_state <= next_state;
        end if;
    end process;

    next_state_logic: process(current_state, X)
    begin
        case current_state is
            when S_0_EVEN =>
                if X = '0' then next_state <= S_0_ODD; else next_state <= S_1_EVEN; end
            when S_0_ODD =>
                if X = '0' then next_state <= S_0_EVEN; else next_state <= S_1_ODD; end
            when S_1_EVEN =>
                if X = '0' then next_state <= S_1_ODD; else next_state <= S_2_EVEN; end
            when S_1_ODD =>
                if X = '0' then next_state <= S_1_EVEN; else next_state <= S_2_ODD; end
            when S_2_EVEN =>
                if X = '0' then next_state <= S_2_ODD; else next_state <= S_3_EVEN; end
            when S_2_ODD =>
                if X = '0' then next_state <= S_2_EVEN; else next_state <= S_3_ODD; end
```

```
            when S_3_EVEN =>
                if X = '0' then next_state <= S_3_ODD; else next_state <= S_0_EVEN; end
            when S_3_ODD =>
                if X = '0' then next_state <= S_3_EVEN; else next_state <= S_0_ODD; end
            when others =>
                next_state <= S_0_EVEN;
        end case;
    end process;

    output_logic: process(current_state)
    begin
        case current_state is
            when S_0_ODD => Z <= '1';
            when others  => Z <= '0';
        end case;
    end process;
end Behavioral;
```

9.2 Mealy Architecture

```
-- sequential_circuit_mealy.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sequential_circuit_mealy is
    Port (
        X   : in  STD_LOGIC;
        Clk : in  STD_LOGIC;
        Z   : out STD_LOGIC
    );
end sequential_circuit_mealy;

architecture Behavioral of sequential_circuit_mealy is
    type state_type is (
        S_0_EVEN, S_0_ODD, S_1_EVEN, S_1_ODD,
        S_2_EVEN, S_2_ODD, S_3_EVEN, S_3_ODD
    );
    signal current_state, next_state : state_type := S_0_EVEN;
begin
    state_register: process(Clk)
    begin
        if rising_edge(Clk) then
            current_state <= next_state;
        end if;
    end process;
```

```
    mealy_logic: process(current_state, X)
    begin
        Z <= '0';
        case current_state is
            when S_0_EVEN =>
                if X = '0' then next_state <= S_0_ODD; Z <= '1';
                else next_state <= S_1_EVEN; Z <= '0'; end if;
            when S_0_ODD =>
                if X = '0' then next_state <= S_0_EVEN; Z <= '0';
                else next_state <= S_1_ODD; Z <= '0'; end if;
            when S_1_EVEN =>
                if X = '0' then next_state <= S_1_ODD; Z <= '0';
                else next_state <= S_2_EVEN; Z <= '0'; end if;
            when S_1_ODD =>
                if X = '0' then next_state <= S_1_EVEN; Z <= '0';
                else next_state <= S_2_ODD; Z <= '0'; end if;
            when S_2_EVEN =>
                if X = '0' then next_state <= S_2_ODD; Z <= '0';
                else next_state <= S_3_EVEN; Z <= '0'; end if;
            when S_2_ODD =>
                if X = '0' then next_state <= S_2_EVEN; Z <= '0';
                else next_state <= S_3_ODD; Z <= '0'; end if;
            when S_3_EVEN =>
                if X = '0' then next_state <= S_3_ODD; Z <= '0';
                else next_state <= S_0_EVEN; Z <= '0'; end if;
            when S_3_ODD =>
                if X = '0' then next_state <= S_3_EVEN; Z <= '0';
                else next_state <= S_0_ODD; Z <= '1'; end if;
            when others =>
                next_state <= S_0_EVEN; Z <= '0';
        end case;
    end process;
end Behavioral;
```